

50.007 Machine Learning

Lecture 4

Regression

Machine Learning



Task



Performance

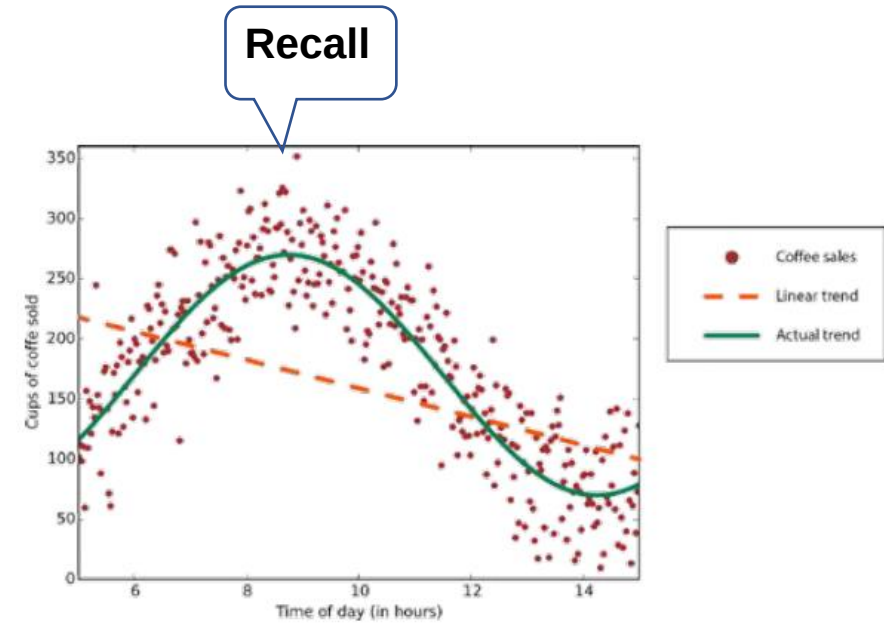


Experience

Algorithms that improve their **performance** at some **task** with **experience**
– Tom Mitchell (1998)

Linear Regression

Machine Learning
 > Supervised Learning
 > Classification
 > Regression



- **Task.** Find function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ such that $y \approx f(x; \theta)$
- **Experience.** Training data $(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})$
- **Performance.** Prediction error $(y - f(x; \theta))$ on test data

Linear Regression

Training data

$$\mathcal{S}_n = \{ (x^{(t)}, y^{(t)}) \mid t = 1, \dots, n \}$$

- Features/Inputs $x^{(t)} = (x_1^{(t)}, \dots, x_d^{(t)})^\top \in \mathbb{R}^d$
- Response/Output $y^{(t)} \in \mathbb{R}$

Linear Regression

Model (or Hypothesis Class) F

Each f is a *predictor*
or *hypothesis*

Set of *linear* functions $f: \mathbb{R}^d \rightarrow \mathbb{R}$

$$f(x; \theta, \theta_0) = \theta \cdot x = \theta_d x_d + \cdots + \theta_1 x_1 + \theta_0 = \theta^\top x + \theta_0$$

Model Parameters

$$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$$

Least Square Loss

Loss Function $\text{Loss}(z) = \frac{1}{2} z^2 = \frac{1}{2} (y^{(t)} - (\theta \cdot x^{(t)}))^2$ Squared error.
Penalize big errors more heavily.
CONVEX!!

Empirical Risk/Training Error

$$R_1(\theta; x, y) = \text{Loss}(y^{(t)} - (\theta \cdot x^{(t)}))$$

Point error

$$R_n(\theta; \mathcal{S}_n) = \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_n} R_1(\theta; x, y)$$

Average error

$$= \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_n} \frac{1}{2} (y^{(t)} - (\theta \cdot x^{(t)}))^2$$

The training error is the average of the point errors.

Risk = “Expected Error”
Empirical = “of the Data”

Linear Regression

- Empirical Risk and Least Squares Error

Training Error

$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \text{Loss}(y^{(t)} - \theta \cdot x^{(t)}) = \frac{1}{n} \sum_{t=1}^n (y^{(t)} - \theta \cdot x^{(t)})^2 / 2$$

Training Algorithm

Find predictor $\hat{f} \in F$ that minimizes $R_n(\theta)$

The test error and training error can be different.

Linear Regression

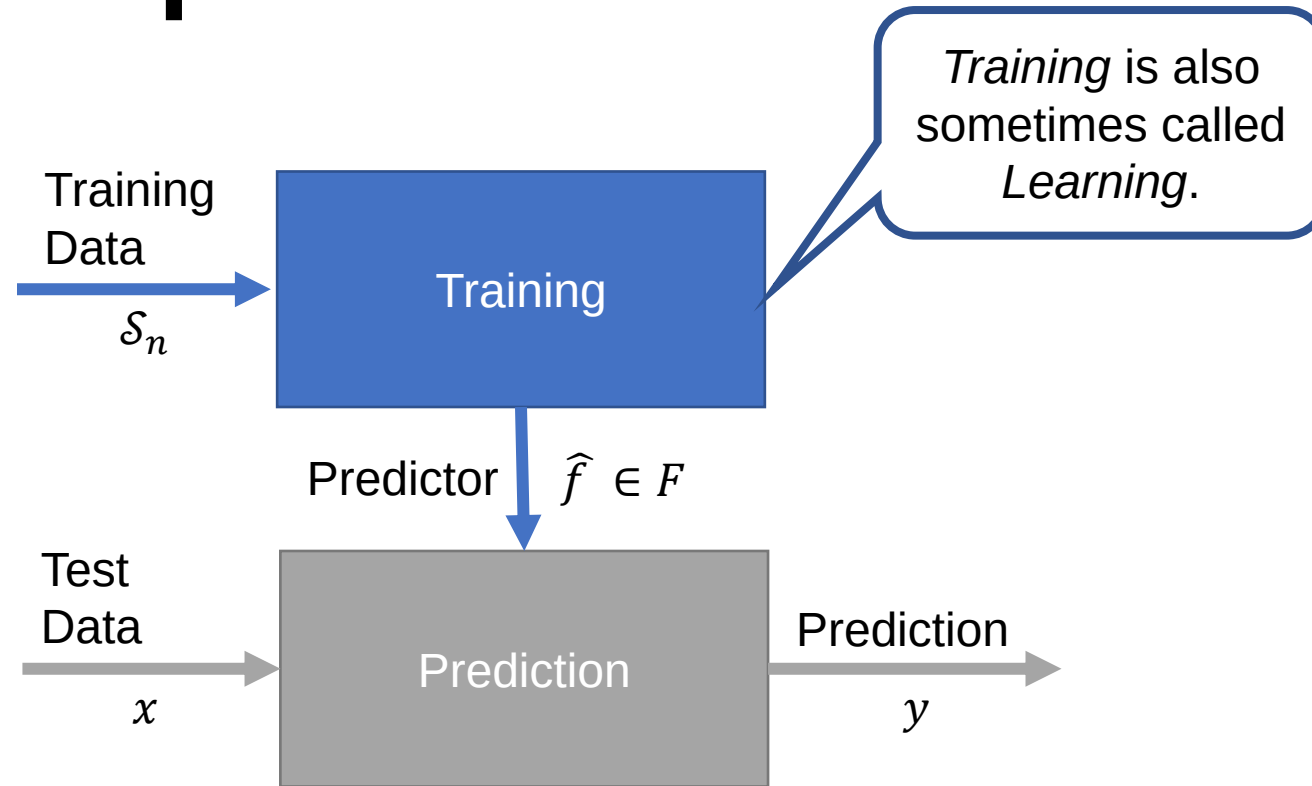
- Empirical Risk and Least Squares Error

Test Error

$$R_{n'}^{test}(\theta) = \frac{1}{|S_{n'}|} \sum_{t=n+1}^{n+n'} (y^{(t)} - \theta \cdot x^{(t)})^2 / 2$$

Given a predictor $\hat{f}$, we use the test error to measure how well it generalizes to new data.

Training and prediction



Assumption. Test data and training data are **identically distributed**.

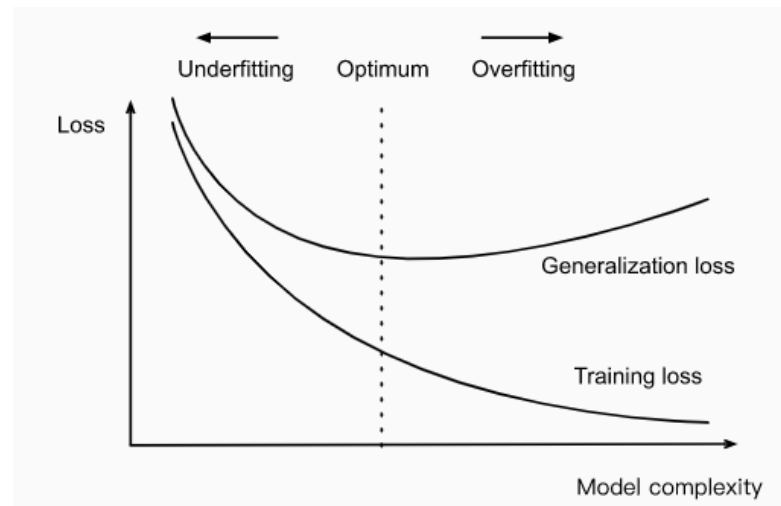
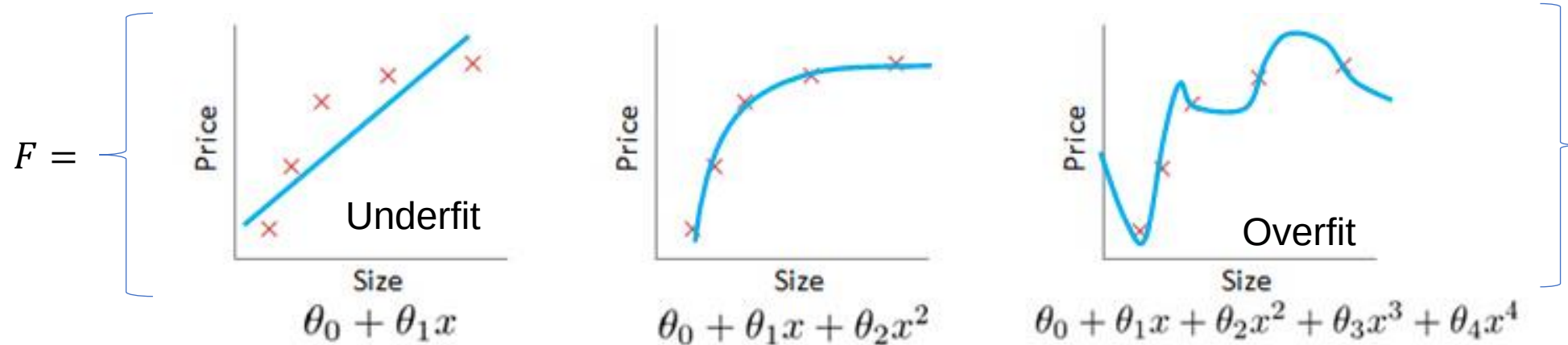
Generalization

The goal of machine learning is to find a predictor $\hat{f} \in F$ that **generalizes** well, i.e. that predicts well on test data $\mathcal{S}_{n'}$.

Types of errors

- **Estimation error (variance):**
 - Caused due to small/noisy dataset ↴
- **Structural error (bias):**
 - Caused due to small set of predictors, F ↴ *choice of predictors/models*
- The two errors are cyclic, e.g. attempting to reduce structural errors with noisy data can lead to high estimation error.
- We need to find a balance between the two errors.

Under and Overfitting



Model Selection

Overfitting. If model F is too big, then $\hat{f} \in F$ performs well on training data, but poorly on test data.

Underfitting. If model F is too small, then $\hat{f} \in F$ performs poorly on training data, and poorly on test data.

Finding a model with the right size is called **model selection**.

Optimization

Gradient Descent

- Use **gradient descent** to minimize $R_n(\theta)$

$$\nabla_{\theta} R_n(\theta) = \left[\frac{\partial R_n(\theta)}{\partial \theta_1}, \dots, \frac{\partial R_n(\theta)}{\partial \theta_d} \right]^T$$

- Positive gradient points in the direction where $R_n(\theta)$ **increases**.
- Need to update the weight in the **opposite direction**.

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \nabla_{\theta} R_n(\theta)_{\theta=\theta^{(k)}}$$

↓
learning rate
of model

Gradient Descent

- Empirical Risk

$$R_n(\theta) = \frac{1}{n} \sum_{t=1}^n \text{Loss}(y^{(t)} - \theta \cdot x^{(t)}) = \frac{1}{n} \sum_{t=1}^n (y^{(t)} - \theta \cdot x^{(t)})^2 / 2$$

- Partial Derivative

$$\nabla_{\theta} (y^{(t)} - \theta \cdot x^{(t)})^2 / 2 = (y^{(t)} - \theta \cdot x^{(t)}) \nabla_{\theta} (y^{(t)} - \theta \cdot x^{(t)}) = -(y^{(t)} - \theta \cdot x^{(t)}) x^{(t)}$$

- Update of weight

$$\begin{aligned} \theta^{(k+1)} &= \theta^{(k)} - \eta_k \nabla_{\theta} \frac{1}{2} (y^{(k)} - \theta^{(k)} \cdot x^{(k)})^2 \\ &= \theta^{(k)} - \eta_k (-x^{(k)}) (y^{(k)} - \theta^{(k)} \cdot x^{(k)}) \\ &= \theta^{(k)} + \eta_k (y^{(k)} - \theta^{(k)} \cdot x^{(k)}) x^{(k)} \end{aligned}$$

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \nabla_{\theta} R_n(\theta)_{\theta=\theta^{(k)}}$$

Update rule for batch
gradient descent

Update rule for stochastic
gradient descent

Stochastic Gradient Descent

1. Initialize the **weight** ($\theta^{(0)} = 0$).

2. Select $t \in \{1, \dots, n\}$ at random

$$\theta^{(k+1)} = \theta^{(k)} + \eta_k (y^{(t)} - \theta \cdot x^{(t)}) x^{(t)}$$

3. Repeat Step (2) until stopping criterion is met.
(e.g. when improvement in $R_n(\theta)$ is small enough)

* There is no condition to check anymore as compared to linear classification

Closed Form Solution

- Minimize empirical risk directly by setting gradient to zero.

$$\begin{aligned}\nabla R_n(\theta)_{\theta=\hat{\theta}} &= \frac{1}{n} \sum_{t=1}^n \nabla_{\theta} \left\{ (y^{(t)} - \theta \cdot x^{(t)})^2 / 2 \right\}_{\theta=\hat{\theta}} \\ &= \frac{1}{n} \sum_{t=1}^n \{ -(y^{(t)} - \hat{\theta} \cdot x^{(t)}) x^{(t)} \} \\ &= -\frac{1}{n} \sum_{t=1}^n y^{(t)} x^{(t)} + \frac{1}{n} \sum_{t=1}^n (\hat{\theta} \cdot x^{(t)}) x^{(t)} \\ &= -\underbrace{\frac{1}{n} \sum_{t=1}^n y^{(t)} x^{(t)}}_{=b} + \underbrace{\frac{1}{n} \sum_{t=1}^n x^{(t)} (x^{(t)})^T}_{=A} \hat{\theta} \\ &= -b + A\hat{\theta} = 0\end{aligned}$$

* From linear algebra
1. Take partial derivative
2. Set it to zero
3. Solve for $\hat{\theta}$.

$$\longrightarrow y = Ax + b$$

Closed Form Solution

$$\begin{aligned} A\hat{\theta} - b &= 0 \\ A\hat{\theta} &= b \\ \hat{\theta} &= \frac{b}{A} \end{aligned}$$

- If A is invertible ($n \geq d$), we can find the weight as below

$$\hat{\theta} = A^{-1}b.$$

- Where, $b = \frac{1}{n} X^T \bar{y}$, $A = \frac{1}{n} X^T X$

$\begin{array}{cc} \downarrow & \downarrow \\ d \times n & n \times 1 \end{array} \quad \begin{array}{cc} \downarrow & \downarrow \\ d \times n & n \times d \end{array}$

* There must be much more training examples compared to # of features $\because$ if not it will be difficult to invert & the model will not be able to learn sufficiently.

Regularization

Ridge Regression

Some features may be irrelevant to the predictor but we cannot ensure that the weight = 0, $\therefore$ we may choose to ignore by picking a simple model.

Height Weight Age Temp. on Mars

$$y \approx \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d$$

For simplicity, we ignore θ_0 .

How do we ensure that $\theta_i = 0$ when feature x_i is irrelevant?

Pick simplest model that explains data $\rightarrow$ **generalization**

Ridge Regression

when $\lambda = 0$, the model focus is to fit the data
↑ $\lambda > 0$, the model focus is to simplify so that
it prevents overfitting & underfitting
on training data & test data.

Add a penalty term to empirical risk

$$J_{n,\lambda}(\theta) = \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_n} \frac{1}{2} (y - \theta^\top x)^2 + \frac{\lambda}{2} \|\theta\|^2$$

Pressure to fit data

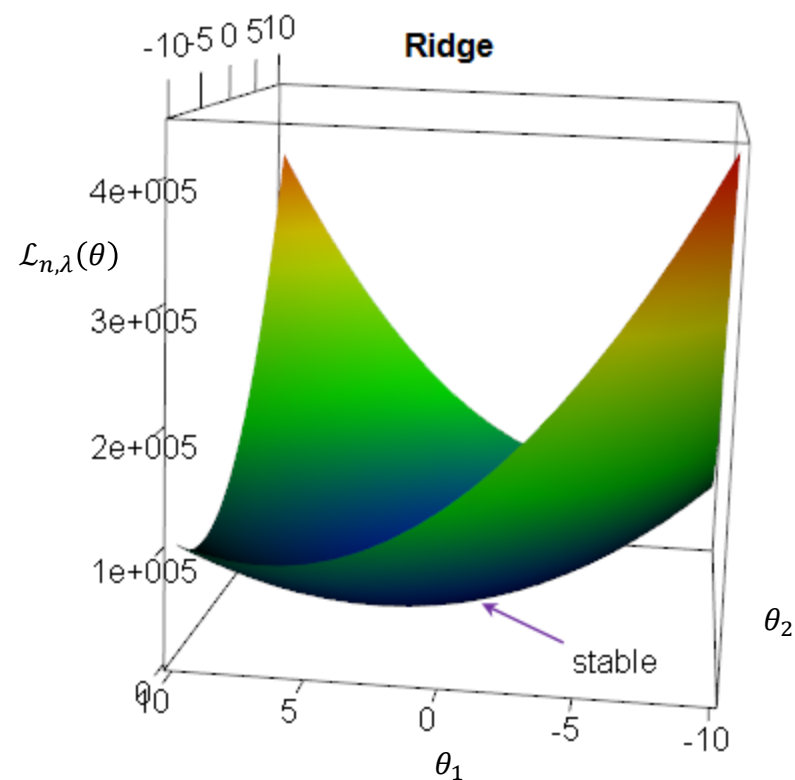
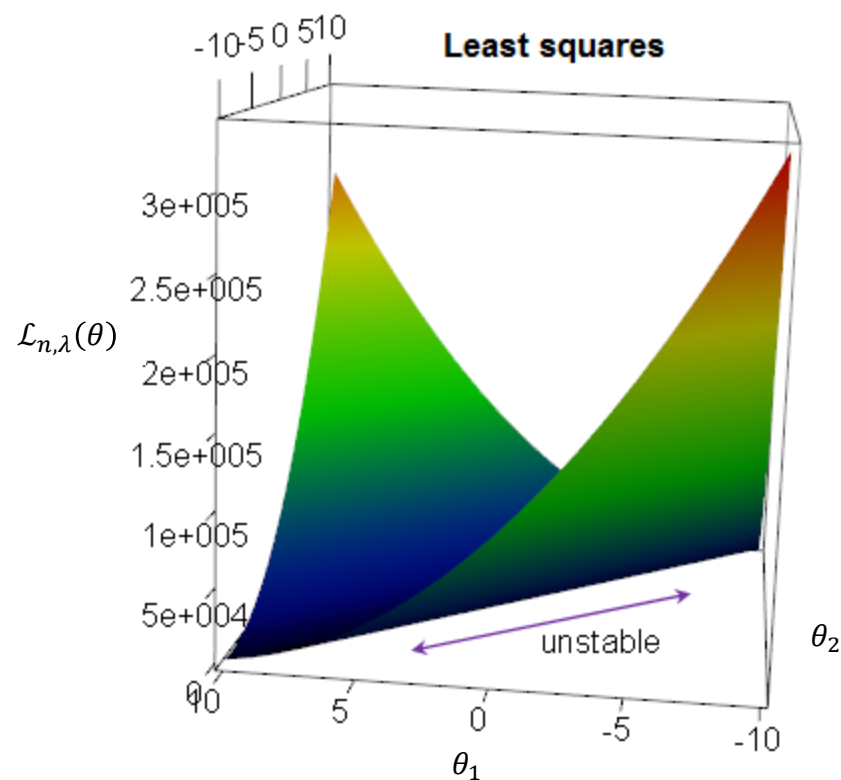
Pressure to
simplify model

Regularization
parameter $\lambda \geq 0$

Regularizer

→ To simplify our
model as much as
possible by lowering
λ value.

Ridge Regression



Training Algorithm (SGD)

Ridge Regression $J_{n,\lambda}(\theta) = \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_n} \frac{1}{2} (y - \theta^\top x)^2 + \frac{\lambda}{2} \|\theta\|^2$

Gradient $\nabla J_{n,\lambda}(\theta) = \nabla_{\theta} \left\{ \frac{\lambda}{2} \|\theta\|^2 + (y^{(t)} - \theta \cdot x^{(t)})^2 / 2 \right\}_{|\theta=\theta^{(k)}}$

$$\nabla J_{n,\lambda}(\theta) = \lambda \theta^{(k)} - (y^{(t)} - \theta^{(k)} \cdot x^{(t)}) x^{(t)}$$

Gradient Descent

$$\begin{aligned} \theta^{(k+1)} &= \theta^{(k)} - \eta_k \nabla J_{n,\lambda}(\theta) = \theta^{(k)} - \eta_k [\lambda \theta^{(k)} - (y^{(t)} - \theta^{(k)} \cdot x^{(t)}) x^{(t)}] \\ \theta^{(k+1)} &= (1 - \lambda \eta_k) \theta^{(k)} + \eta_k (y^{(t)} - \theta^{(k)} \cdot x^{(t)}) x^{(t)} \\ &= \theta^{(k)} - \eta_k \lambda \theta^{(k)} + \eta_k (y^{(t)} - \theta^{(k)} \cdot x^{(t)}) x^{(t)} \\ &= \theta^{(k)} (1 - \lambda \eta_k) + \eta_k (y^{(t)} - \theta^{(k)} \cdot x^{(t)}) x^{(t)} \end{aligned}$$

Without regularization,
i.e. $\lambda = 0$, this shrinkage
factor equals 1.

* closed form solution

Training Algorithm (Exact Solution)

Ridge Regression

$$J_{n,\lambda}(\theta) = \frac{1}{n} \sum_{(x,y) \in \mathcal{S}_n} \frac{1}{2} (y - \theta^\top x)^2 + \frac{\lambda}{2} \|\theta\|^2$$

} written in
matrix form

Gradient

$$\nabla J_{n,\lambda}(\theta) = \lambda \theta + \frac{1}{n} (X^\top X) \theta - \frac{1}{n} X^\top Y$$

Exact Solution

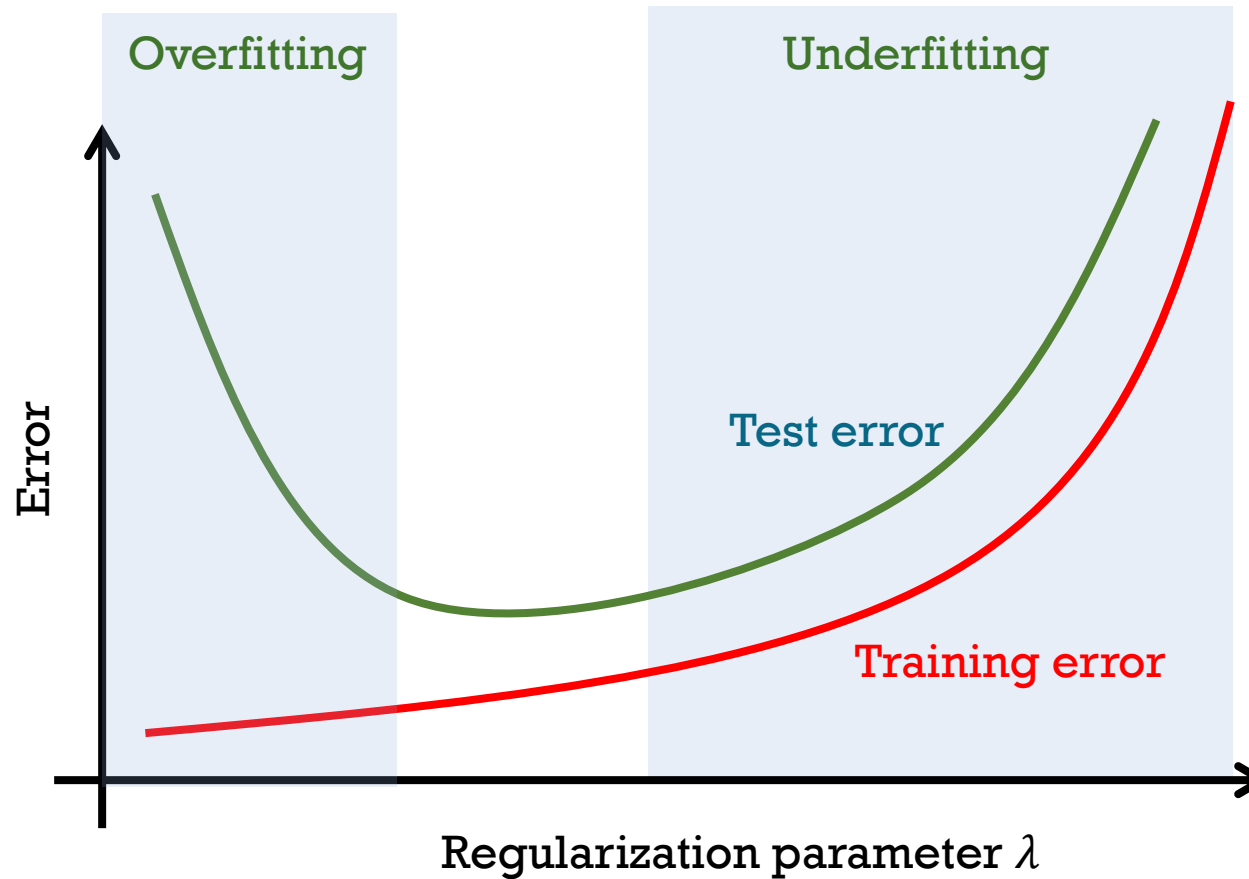
$$\nabla J_{n,\lambda}(\hat{\theta}) = 0 \quad \Leftrightarrow \quad \lambda \hat{\theta} + \frac{1}{n} (X^\top X) \hat{\theta} = \frac{1}{n} X^\top Y$$

$$\Leftrightarrow \quad \hat{\theta} = (n\lambda I + X^\top X)^{-1} X^\top Y$$

→ note: there is an
identity matrix
in the eqn.

This matrix is always
invertible when $\lambda > 0$.

Effect of Regularization



Picking Hyperparameters

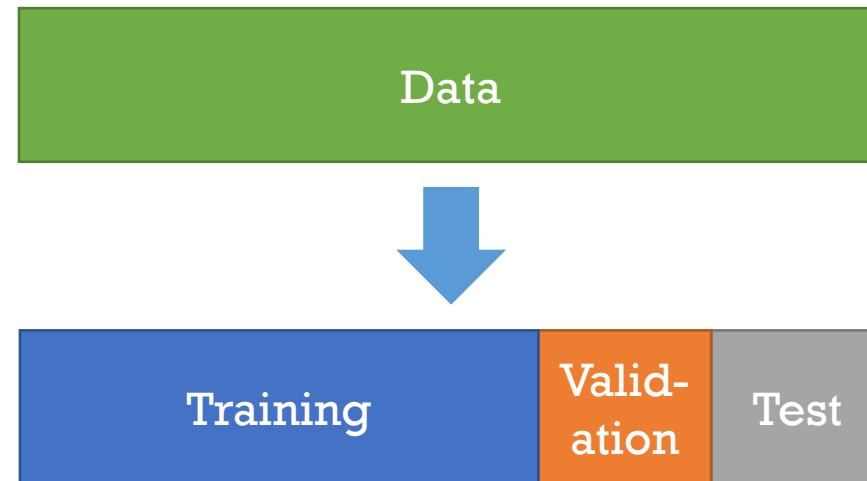
- The regularization parameter λ is an example of a *hyperparameter*, which affects the model complexity.
- We don't usually have access to the test data.
How do we know if the value of λ minimizes the test error?
- The solution is to create a *validation data set*, as a proxy to the test data, and to compute the *validation error*.

↓
i.e. 2% of the test data. ↗

Validation Set

Split the data into

- **Test set** $\mathcal{S}_{n'}$
For evaluating, reporting performance at the end
- **Training set** $\mathcal{S}_n$
For training optimal parameters in a model
- **Validation set** $\mathcal{S}_{\text{val}}$
For model selection, e.g. picking λ in ridge regression.
Acts as a proxy for test set.



Validation Error

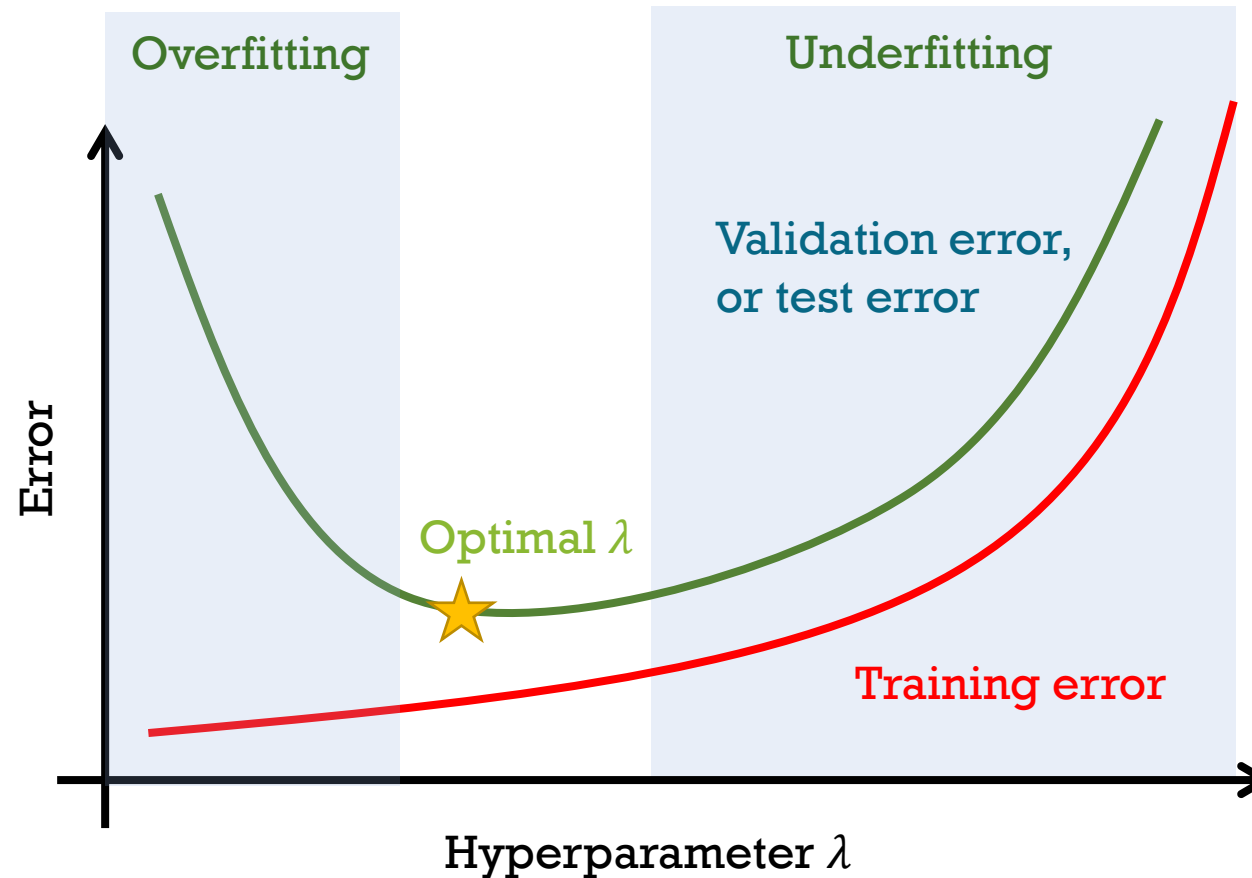
The *validation error* is the test error applied to the validation set.

Example. Ridge Regression

Test error $\mathcal{R}(\hat{\theta}; \mathcal{S}_{n'}) = \frac{1}{|\mathcal{S}_{n'}|} \sum_{(x,y) \in \mathcal{S}_{n'}} \frac{1}{2} (y - \hat{\theta}^\top x)^2$

Validation error $\mathcal{R}(\hat{\theta}; \mathcal{S}_{\text{val}}) = \frac{1}{|\mathcal{S}_{\text{val}}|} \sum_{(x,y) \in \mathcal{S}_{\text{val}}} \frac{1}{2} (y - \hat{\theta}^\top x)^2$

Model Selection



Learning Objectives

- What is linear regression and how to learn a linear regression model?
- What is ridge regression and how is it different from linear regression?
- What is regularization and why do we need to do regularization?